Advanced Computer Architecture LAB
Autumn Semester 2025-26
Lab Assignment 3

Name – Soham Chakraborty
Roll No – 22CS02002

Ans-1. i) Output of test-math using ‘sim-outorder’

```
goku@DESKTOP-RHVH0B7:~/SimpleScalar/Assignment_3$ alias soo="$IDIR/simplesim-3.0/sim-outorder"
goku@DESKTOP-RHVH0B7:~/SimpleScalar/Assignment_3$ alias pv=/home/SimpleScalar/simplesim-3.0/pipeview.pl
goku@DESKTOP-RHVH0B7:~/SimpleScalar/Assignment_3$ alias pv="$IDIR/simplesim-3.0/pipeview.pl"
goku@DESKTOP-RHVH0B7:~/SimpleScalar/Assignment_3$ soo -config config_a.cfg -ptrace config_a.trc 0:1024 -redir:sim sim_configa.out ./t
est-math
pow(12.0, 2.0) == 144.000000
pow(10.0, 3.0) == 1000.000000
pow(10.0, -3.0) == 0.001000
str: 123.456
x: 123.000000
str: 123.456
x: 123.456000
str: 123.456
x: 123.456000
123.456 123.456000 123 1000
sinh(2.0) = 3.62686
sinh(3.0) = 10.01787
h=3.60555
atan2(3,2) = 0.98279
pow(3.60555,4.0) = 169
169 / exp(0.98279 * 5) = 1.24102
3.93117 + 5*log(3.60555) = 10.34355
cos(10.34355) = -0.6068,  sin(10.34355) = -0.79486
x     0.5x
x0.5     x
x   0.5x
-1e-17 != -1e-17 Worked!
goku@DESKTOP-RHVH0B7:~/SimpleScalar/Assignment_3$
```

Ans – 1. j) Output of ‘sim_configa.out’

```
goku@DESKTOP-RHVH0B7:~/SimpleScalar/Assignment_3$ cat sim_configa.out
sim-outorder: SimpleScalar/PISA Tool Set version 3.0 of August, 2003.
Copyright (c) 1994-2003 by Todd M. Austin, Ph.D. and SimpleScalar, LLC.
All Rights Reserved. This version of SimpleScalar is licensed for academic
non-commercial use.  No portion of this work may be used by any commercial
entity, or for any commercial purpose, without the prior written permission
of SimpleScalar, LLC (info@simplescalar.com).

sim: command line: /home/goku/installn_files/simplesim-3.0/sim-outorder -config config_a.cfg -ptrace config_a.trc 0:16

sim: simulation started @ Thu Aug 21 16:27:54 2025, options follow:

sim-outorder: This simulator implements a very detailed out-of-order issue
superscalar processor with a two-level memory system and speculative
execution support.  This simulator is a performance simulator, tracking the
latency of all pipeline operations.

# -config                      # load configuration from a file
# -dumpconfig                  # dump configuration to a file
# -h                     false # print help message
# -v                     false # verbose operation
# -d                     false # enable debug message
# -i                     false # start in Dlite debugger
-seed                        1 # random number generator seed (0 for timer seed)
# -q                     false # initialize and terminate immediately
# -chkpt                 <null> # restore EIO trace execution from <fname>
# -redir:sim     sim_configa.out # redirect simulator output to file (non-interactive only)
# -redir:prog          <null> # redirect simulated program output to file
-nice                        0 # simulator scheduling priority
-max:inst                    0 # maximum number of inst's to execute
-fastfwd                     0 # number of insts skipped before timing starts
# -ptrace        config_a.trc 0:1024 # generate pipetrace, i.e., <fname|stdout|stderr> <range>
-fetch:ifqsize               1 # instruction fetch queue size (in insts)
-fetch:mplat                 3 # extra branch mis-prediction latency
-fetch:speed                 1 # speed of front-end of machine relative to execution core
-bpred                   bimod # branch predictor type {nottaken|taken|perfect|bimod|2lev|comb}
-bpred:bimod      2048 # bimodal predictor config (<table size>)
-bpred:2lev       1 1024 8 # 2-level predictor config (<l1size> <l2size> <hist_size> <xor>)
-bpred:comb       1024 # combining predictor config (<meta_table_size>)
-bpred:ras                   8 # return address stack size (0 for no return stack)
-bpred:btb        512 4 # BTB config (<num_sets> <associativity>)
# -bpred:spec_update       <null> # speculative predictors update in {ID|WB} (default non-spec)
-decode:width                1 # instruction decode B/W (insts/cycle)
-issue:width                 1 # instruction issue B/W (insts/cycle)
-issue:inorder            true # run pipeline with in-order issue
-issue:wrongpath          true # issue instructions down wrong execution paths
-commit:width                4 # instruction commit B/W (insts/cycle)
-ruu:size                    8 # register update unit (RUU) size
-lsq:size                    8 # load/store queue (LSQ) size
-cache:dl1        dl1:128:32:4:l # l1 data cache config, i.e., {<config>|none}
-cache:dl1lat                1 # l1 data cache hit latency (in cycles)
-cache:dl2        ul2:1024:64:4:l # l2 data cache config, i.e., {<config>|none}
-cache:dl2lat                6 # l2 data cache hit latency (in cycles)
-cache:il1        il1:512:32:1:l # l1 inst cache config, i.e., {<config>|dl1|dl2|none}
-cache:il1lat                1 # l1 instruction cache hit latency (in cycles)
-cache:il2               dl2 # l2 instruction cache config, i.e., {<config>|dl2|none}
-cache:il2lat                6 # l2 instruction cache hit latency (in cycles)
-cache:flush             false # flush caches on system calls
-cache:icompress         false # convert 64-bit inst addresses to 32-bit inst equivalents
-mem:lat          18 2 # memory access latency (<first_chunk> <inter_chunk>)
-mem:width                   8 # memory access bus width (in bytes)
-tlb:itlb         itlb:16:4096:4:l # instruction TLB config, i.e., {<config>|none}
-tlb:dtlb         dtlb:32:4096:4:l # data TLB config, i.e., {<config>|none}
-tlb:lat                    30 # inst/data TLB miss latency (in cycles)
-res:ialu                    1 # total number of integer ALU's available
-res:imult                   1 # total number of integer multiplier/dividers available
-res:memport                 1 # total number of memory system ports available (to CPU)
-res:fpalu                   1 # total number of floating point ALU's available
-res:fpmult                  1 # total number of floating point multiplier/dividers available
# -pcstat              <null> # profile stat(s) against text addr's (mult uses ok)
-bugcompat             false # operate in backward-compatible bugs mode (for testing only)

  Pipetrace range arguments are formatted as follows:

    {{@|#}<start>}:{{@|#|+}<end>}

  Both ends of the range are optional, if neither are specified, the entire
  execution is traced.  Ranges that start with a '@' designate an address
  range to be traced, those that start with an '#' designate a cycle count
  range.  All other range values represent an instruction count range.  The
  second argument, if specified with a '+', indicates a value relative
  to the first argument, e.g., 1000:+100 == 1000:1100.  Program symbols may
  be used in all contexts.

    Examples:   -ptrace FOO.trc #0:#1000
                -ptrace BAR.trc @2000:
                -ptrace BLAH.trc :1500
                -ptrace UXXE.trc :
                -ptrace FOOBAR.trc @main:+278

  Branch predictor configuration examples for 2-level predictor:
    Configurations:   N, M, W, X
      N   # entries in first level (# of shift register(s))
      W   width of shift register(s)
      M   # entries in 2nd level (# of counters, or other FSM)
      X   (yes-1/no-0) xor history and address for 2nd level index
    Sample predictors:
      GAg     : 1, W, 2^W, 0
      GAp     : 1, W, M (M > 2^W), 0
      PAg     : N, W, 2^W, 0
      PAp     : N, W, M (M == 2^(N+W)), 0
      gshare  : 1, W, 2^W, 1
  Predictor 'comb' combines a bimodal and a 2-level predictor.

  The cache config parameter <config> has the following format:

    <name>:<nsets>:<bsize>:<assoc>:<repl>

    <name>   - name of the cache being defined
    <nsets>  - number of sets in the cache
    <bsize>  - block size of the cache
    <assoc>  - associativity of the cache
    <repl>   - block replacement strategy, 'l'-LRU, 'f'-FIFO, 'r'-random

    Examples:   -cache:dl1 dl1:4096:32:1:l
                -dtlb dtlb:128:4096:32:r

  Cache levels can be unified by pointing a level of the instruction cache
  hierarchy at the data cache hiearchy using the "dl1" and "dl2" cache
  configuration arguments.  Most sensible combinations are supported, e.g.,

    A unified l2 cache (il2 is pointed at dl2):
      -cache:il1 il1:128:64:1:l -cache:il2 dl2
      -cache:dl1 dl1:256:32:1:l -cache:dl2 ul2:1024:64:2:l

    Or, a fully unified cache hierarchy (il1 pointed at dl1):
      -cache:il1 dl1
      -cache:dl1 ul1:256:32:1:l -cache:dl2 ul2:1024:64:2:l


sim: ** starting performance simulation **
```

```
sim: ** starting performance simulation **

sim: ** simulation statistics **
sim_num_insn                 213598 # total number of instructions committed
sim_num_refs                  56890 # total number of loads and stores committed
sim_num_loads                 34096 # total number of loads committed
sim_num_stores           22794.0000 # total number of stores committed
sim_num_branches              38571 # total number of branches committed
sim_elapsed_time                  1 # total simulation time in seconds
sim_inst_rate            213598.0000 # simulation speed (in insts/sec)
sim_total_insn               216565 # total number of instructions executed
sim_total_refs                57944 # total number of loads and stores executed
sim_total_loads               35031 # total number of loads executed
sim_total_stores         22913.0000 # total number of stores executed
sim_total_branches            38702 # total number of branches executed
sim_cycle                    421199 # total simulation time in cycles
sim_IPC                      0.5071 # instructions per cycle
sim_CPI                      1.9719 # cycles per instruction
sim_exec_BW                  0.5142 # total instructions (mis-spec + committed) per cycle
sim_IPB                      5.5378 # instruction per branch
IFQ_count                    293051 # cumulative IFQ occupancy
IFQ_fcount                   293051 # cumulative IFQ full count
ifq_occupancy                0.6958 # avg IFQ occupancy (insn's)
ifq_rate                     0.5142 # avg IFQ dispatch rate (insn/cycle)
ifq_latency                  1.3532 # avg IFQ occupant latency (cycle's)
ifq_full                     0.6958 # fraction of time (cycle's) IFQ was full
RUU_count                    742051 # cumulative RUU occupancy
RUU_fcount                        8 # cumulative RUU full count
ruu_occupancy                1.7618 # avg RUU occupancy (insn's)
ruu_rate                     0.5142 # avg RUU dispatch rate (insn/cycle)
ruu_latency                  3.4265 # avg RUU occupant latency (cycle's)
ruu_full                     0.0000 # fraction of time (cycle's) RUU was full
LSQ_count                    209888 # cumulative LSQ occupancy
LSQ_fcount                        0 # cumulative LSQ full count
lsq_occupancy                0.4983 # avg LSQ occupancy (insn's)
lsq_rate                     0.5142 # avg LSQ dispatch rate (insn/cycle)
lsq_latency                  0.9692 # avg LSQ occupant latency (cycle's)
lsq_full                     0.0000 # fraction of time (cycle's) LSQ was full
sim_slip                    1218390 # total number of slip cycles
avg_sim_slip                 5.7041 # the average slip between issue and retirement
bpred_bimod.lookups          39565 # total number of bpred lookups
bpred_bimod.updates          38571 # total number of updates
bpred_bimod.addr_hits        34623 # total number of address-predicted hits
bpred_bimod.dir_hits         35114 # total number of direction-predicted hits (includes addr-hits)
bpred_bimod.misses            3457 # total number of misses
bpred_bimod.jr_hits           3520 # total number of address-predicted hits for JR's
bpred_bimod.jr_seen           3542 # total number of JR's seen
bpred_bimod.jr_non_ras_hits.PP           28 # total number of address-predicted hits for non-RAS JR's
bpred_bimod.jr_non_ras_seen.PP           41 # total number of non-RAS JR's seen
bpred_bimod.bpred_addr_rate    0.8976 # branch address-prediction rate (i.e., addr-hits/updates)
bpred_bimod.bpred_dir_rate    0.9104 # branch direction-prediction rate (i.e., all-hits/updates)
bpred_bimod.bpred_jr_rate    0.9938 # JR address-prediction rate (i.e., JR addr-hits/JRs seen)
bpred_bimod.bpred_jr_non_ras_rate.PP    0.6829 # non-RAS JR addr-pred rate (ie, non-RAS JR hits/JRs seen)
bpred_bimod.retstack_pushes         3510 # total number of address pushed onto ret-addr stack
bpred_bimod.retstack_pops           4040 # total number of address popped off of ret-addr stack
bpred_bimod.used_ras.PP             3501 # total number of RAS predictions used
bpred_bimod.ras_hits.PP             3492 # total number of RAS hits
bpred_bimod.ras_rate.PP    0.9974 # RAS prediction rate (i.e., RAS hits/used RAS)
il1.accesses                    235164 # total number of accesses
il1.hits                        219440 # total number of hits
il1.misses                       15724 # total number of misses
il1.replacements                 15214 # total number of replacements
il1.writebacks                       0 # total number of writebacks
il1.invalidations                    0 # total number of invalidations
il1.miss_rate                   0.0669 # miss rate (i.e., misses/ref)
il1.repl_rate                   0.0647 # replacement rate (i.e., repls/ref)
```

```
il1.hits                        219440 # total number of hits
il1.misses                       15724 # total number of misses
il1.replacements                 15214 # total number of replacements
il1.writebacks                       0 # total number of writebacks
il1.invalidations                    0 # total number of invalidations
il1.miss_rate                   0.0669 # miss rate (i.e., misses/ref)
il1.repl_rate                   0.0647 # replacement rate (i.e., repls/ref)
il1.wb_rate                     0.0000 # writeback rate (i.e., wrbks/ref)
il1.inv_rate                    0.0000 # invalidation rate (i.e., invs/ref)
dl1.accesses                     56890 # total number of accesses
dl1.hits                         56343 # total number of hits
dl1.misses                         547 # total number of misses
dl1.replacements                    63 # total number of replacements
dl1.writebacks                      59 # total number of writebacks
dl1.invalidations                    0 # total number of invalidations
dl1.miss_rate                   0.0096 # miss rate (i.e., misses/ref)
dl1.repl_rate                   0.0011 # replacement rate (i.e., repls/ref)
dl1.wb_rate                     0.0010 # writeback rate (i.e., wrbks/ref)
dl1.inv_rate                    0.0000 # invalidation rate (i.e., invs/ref)
ul2.accesses                     16330 # total number of accesses
ul2.hits                         15118 # total number of hits
ul2.misses                        1212 # total number of misses
ul2.replacements                     0 # total number of replacements
ul2.writebacks                       0 # total number of writebacks
ul2.invalidations                    0 # total number of invalidations
ul2.miss_rate                   0.0742 # miss rate (i.e., misses/ref)
ul2.repl_rate                   0.0000 # replacement rate (i.e., repls/ref)
ul2.wb_rate                     0.0000 # writeback rate (i.e., wrbks/ref)
ul2.inv_rate                    0.0000 # invalidation rate (i.e., invs/ref)
itlb.accesses                   235164 # total number of accesses
itlb.hits                       235141 # total number of hits
itlb.misses                         23 # total number of misses
itlb.replacements                    0 # total number of replacements
itlb.writebacks                      0 # total number of writebacks
itlb.invalidations                   0 # total number of invalidations
itlb.miss_rate                  0.0001 # miss rate (i.e., misses/ref)
itlb.repl_rate                  0.0000 # replacement rate (i.e., repls/ref)
itlb.wb_rate                    0.0000 # writeback rate (i.e., wrbks/ref)
itlb.inv_rate                   0.0000 # invalidation rate (i.e., invs/ref)
dtlb.accesses                    56890 # total number of accesses
dtlb.hits                        56880 # total number of hits
dtlb.misses                         10 # total number of misses
dtlb.replacements                    0 # total number of replacements
dtlb.writebacks                      0 # total number of writebacks
dtlb.invalidations                   0 # total number of invalidations
dtlb.miss_rate                  0.0002 # miss rate (i.e., misses/ref)
dtlb.repl_rate                  0.0000 # replacement rate (i.e., repls/ref)
dtlb.wb_rate                    0.0000 # writeback rate (i.e., wrbks/ref)
dtlb.inv_rate                   0.0000 # invalidation rate (i.e., invs/ref)
sim_invalid_addrs                    0 # total non-speculative bogus addresses seen (debug var)
ld_text_base                0x00400000 # program text (code) segment base
ld_text_size                     91744 # program text (code) size in bytes
ld_data_base                0x10000000 # program initialized data segment base
ld_data_size                     13028 # program init'ed `.data' and uninit'ed `.bss' size in bytes
ld_stack_base               0x7fffc000 # program stack segment base (highest address in stack)
ld_stack_size                    16384 # program initial stack size
ld_prog_entry               0x00400140 # program entry point (initial PC)
ld_environ_base             0x7fff8000 # program environment base address address
ld_target_big_endian                 0 # target executable endian-ness, non-zero if big endian
mem.page_count                      33 # total number of pages allocated
mem.page_mem                      132k # total size of memory pages allocated
mem.ptab_misses                     37 # total first level page table misses
mem.ptab_accesses              1932368 # total page table accesses
mem.ptab_miss_rate              0.0000 # first level page table miss rate

goku@DESKTOP-RHVH0B7:~/SimpleScalar/Assignment_3$
```

Ans – Q1) Configuration –

- Random number generator seed = 1
- Instruction Fetch Queue Size = 1
- Extra Branch mis-prediction latency = 3
- Fetch speed = 3
- Branch Predictor type = Bimodal
- Branch Predictor Table size = 2048
- Decode Width = 1
- Issue Width = 1
- Run pipeline with inorder issue = **true**
- Commit width = 4
- Register update unit (ruu) size = 8
- Load/Store queue size = 8
- L1 cache hit latency = 1 cycle
- L2 cache hit latency = 6 cycles
- L1 size : L2 size = 1 : 1024
- Memory access bus width = 8
- Instruction/Data TLB miss latency = 30 cycles
- Integer ALU's available = 1
- Integer multiplers/dividers available = 1
- Memory System ports available = 1
- Floating point ALU's available = 1
- Floating point multiplier/dividers available = 1

Ans – Q2) sim_IPC = 0.5071 # instructions per cycle

Ans – Q3) Output of ‘pv config_a.trc | less’

```
@ 44
ag = `0x00400168: sll       r2,r16,2'

 [IF]      [DA]      [EX]      [WB]      [CT]
  ag        af        ab*!      aa
                               ac
                               ad
                               ae

@ 45
ah = `0x00400170: addu      r3,r3,r2'

 [IF]      [DA]      [EX]      [WB]      [CT]
  ah        ag        ab*!      aa
                      af        ac
                               ad
                               ae
```

We observe that at clock cycle **68** ah is in [EX] and ag is in [WB]. Hence, forwarding is implemented.

```
 [IF]      [DA]      [EX]      [WB]      [CT]
  ah        ag        ab*!      aa
                               ac
                               ad
                               ae
                               af

@ 67
ai = `0x00400178: addu      r18,r0,r3'

 [IF]      [DA]      [EX]      [WB]      [CT]
  ai        ah        ag        aa
                               ab
                               ac
                               ad
                               ae
                               af

@ 68

 [IF]      [DA]      [EX]      [WB]      [CT]
            ai        ah        af        aa
                               ag        ab
                               ac
                               ad
                               ae

@ 69

 [IF]      [DA]      [EX]      [WB]      [CT]
                      ai        ah        af
                               ag
:|
```

Ans – 2) a) Program with intentional RAW hazards. fibonacci.c.

```
goku@DESKTOP-RHVH0B7:~/SimpleScalar/Assignment_3$ cat fibonacci.c
#include <stdio.h>

int main(){
        int i,a=0,b=1,c,n=20;
        printf("Fibonacci Series:\n\n");
        for(i=1;i<=n;i++){
                c=a+b;
                a=b;
                b=c;
                printf("%d: %d\n",i,a);
        }
        return 0;
}
```

b) Compilation using -O0

```
goku@DESKTOP-RHVH0B7:~/SimpleScalar/Assignment_3$ $IDIR/bin/sslittle-na-sstrix-gcc -O0 -o O0 fibonacci.c
goku@DESKTOP-RHVH0B7:~/SimpleScalar/Assignment_3$ soo -config config_a.cfg -ptrace O0.trc 0:4000 -redir:sim sim_O0.out ./O0
Fibonacci Series:

1: 1
2: 1
3: 2
4: 3
5: 5
6: 8
7: 13
8: 21
9: 34
10: 55
11: 89
12: 144
13: 233
14: 377
15: 610
16: 987
17: 1597
18: 2584
19: 4181
20: 6765
```

c) Visualizing stalls using ‘pv O0.trc | less’

- ‘ab’ and ‘ac’ are stalled in Cycle 33.
- ‘aa’ and ‘ab’ are stalled in Cycle 35.
- ‘aa’ and ‘ab’ are stalled again In Cycle 36.

There are many more stalls further but here we are considering only a select few of them.

```
@ 33
ab = `0x00400140: [internal ld/st]'
ac = `0x00400148: lui       r28,0x1001'

 [IF]      [DA]      [EX]      [WB]      [CT]
  ac        aa
            ab

@ 34

 [IF]      [DA]      [EX]      [WB]      [CT]
  ac        ab        aa+

@ 35
ad = `0x00400150: addiu     r28,r28,-27040'

 [IF]      [DA]      [EX]      [WB]      [CT]
  ad        ac        ab*!      aa

@ 36
ae = `0x00400158: addiu     r17,r29,4'

 [IF]      [DA]      [EX]      [WB]      [CT]
  ae        ad        ab*!      aa
                      ac

@ 37

 [IF]      [DA]      [EX]      [WB]      [CT]
            ae        ab*!      aa
                      ad        ac

:
```

d) Optimization using -O2 and -O3 flags

```
goku@DESKTOP-RHVH0B7:~/SimpleScalar/Assignment_2$ $IDIR/bin/sslittle-na-sstrix-gcc -O2 -o O2 fibonacci.c
goku@DESKTOP-RHVH0B7:~/SimpleScalar/Assignment_2$ soo -config config_a.cfg -ptrace O2.trc 0:4000 -redir:sim sim_O2.out ./O2
Fibonacci Series:

1: 1
2: 1
3: 2
4: 3
5: 5
6: 8
7: 13
8: 21
9: 34
10: 55
11: 89
12: 144
13: 233
14: 377
15: 610
16: 987
17: 1597
18: 2584
19: 4181
20: 6765
goku@DESKTOP-RHVH0B7:~/SimpleScalar/Assignment_2$ $IDIR/bin/sslittle-na-sstrix-gcc -O3 -o O3 fibonacci.c
goku@DESKTOP-RHVH0B7:~/SimpleScalar/Assignment_2$ soo -config config_a.cfg -ptrace O3.trc 0:4000 -redir:sim sim_O3.out ./O3
Fibonacci Series:

1: 1
2: 1
3: 2
4: 3
5: 5
6: 8
7: 13
8: 21
9: 34
10: 55
11: 89
12: 144
13: 233
14: 377
15: 610
16: 987
17: 1597
18: 2584
19: 4181
20: 6765
goku@DESKTOP-RHVH0B7:~/SimpleScalar/Assignment_2$
```

e) IPC for O0 = 0.4756
IPC for O2 = 0.4778
IPC for O3 = 0.4778

```
sim: ** simulation statistics **
sim_num_insn                 46468 # total number of instructions committed
sim_num_refs                 14699 # total number of loads and stores committed
sim_num_loads                 8123 # total number of loads committed
sim_num_stores           6576.0000 # total number of stores committed
sim_num_branches             10107 # total number of branches committed
sim_elapsed_time                 1 # total simulation time in seconds
sim_inst_rate           46468.0000 # simulation speed (in insts/sec)
sim_total_insn               46874 # total number of instructions executed
sim_total_refs               14822 # total number of loads and stores executed
sim_total_loads               8237 # total number of loads executed
sim_total_stores         6585.0000 # total number of stores executed
sim_total_branches           10171 # total number of branches executed
sim_cycle                    97707 # total simulation time in cycles
sim_IPC                     0.4756 # instructions per cycle
sim_CPI                     2.1027 # cycles per instruction
sim_exec_BW                 0.4797 # total instructions (mis-spec + committed) per cycle
sim_IPB                     4.5976 # instruction per branch
IFQ_count                    67519 # cumulative IFQ occupancy
IFQ_fcount                   67519 # cumulative IFQ full count
ifq_occupancy               0.6910 # avg IFQ occupancy (insn's)
ifq_rate                    0.4797 # avg IFQ dispatch rate (insn/cycle)
ifq_latency                 1.4404 # avg IFQ occupant latency (cycle's)
ifq_full                    0.6910 # fraction of time (cycle's) IFQ was full
RUU_count                   167630 # cumulative RUU occupancy
RUU_fcount                       0 # cumulative RUU full count
ruu_occupancy               1.7156 # avg RUU occupancy (insn's)
ruu_rate                    0.4797 # avg RUU dispatch rate (insn/cycle)
ruu_latency                 3.5762 # avg RUU occupant latency (cycle's)
ruu_full                    0.0000 # fraction of time (cycle's) RUU was full
LSQ_count                    53226 # cumulative LSQ occupancy
LSQ_fcount                       0 # cumulative LSQ full count
lsq_occupancy               0.5448 # avg LSQ occupancy (insn's)
lsq_rate                    0.4797 # avg LSQ dispatch rate (insn/cycle)
lsq_latency                 1.1355 # avg LSQ occupant latency (cycle's)
lsq_full                    0.0000 # fraction of time (cycle's) LSQ was full
```

```
sim: ** simulation statistics **
sim_num_insn                 46243 # total number of instructions committed
sim_num_refs                 14439 # total number of loads and stores committed
sim_num_loads                 7944 # total number of loads committed
sim_num_stores           6495.0000 # total number of stores committed
sim_num_branches             10084 # total number of branches committed
sim_elapsed_time                 1 # total simulation time in seconds
sim_inst_rate           46243.0000 # simulation speed (in insts/sec)
sim_total_insn               46627 # total number of instructions executed
sim_total_refs               14563 # total number of loads and stores executed
sim_total_loads               8057 # total number of loads executed
sim_total_stores         6506.0000 # total number of stores executed
sim_total_branches           10147 # total number of branches executed
sim_cycle                    96775 # total simulation time in cycles
sim_IPC                     0.4778 # instructions per cycle
sim_CPI                     2.0927 # cycles per instruction
sim_exec_BW                 0.4818 # total instructions (mis-spec + committed) per cycle
sim_IPB                     4.5858 # instruction per branch
IFQ_count                    67186 # cumulative IFQ occupancy
IFQ_fcount                   67186 # cumulative IFQ full count
ifq_occupancy               0.6942 # avg IFQ occupancy (insn's)
ifq_rate                    0.4818 # avg IFQ dispatch rate (insn/cycle)
ifq_latency                 1.4409 # avg IFQ occupant latency (cycle's)
ifq_full                    0.6942 # fraction of time (cycle's) IFQ was full
RUU_count                   166827 # cumulative RUU occupancy
RUU_fcount                       0 # cumulative RUU full count
ruu_occupancy               1.7239 # avg RUU occupancy (insn's)
ruu_rate                    0.4818 # avg RUU dispatch rate (insn/cycle)
ruu_latency                 3.5779 # avg RUU occupant latency (cycle's)
ruu_full                    0.0000 # fraction of time (cycle's) RUU was full
LSQ_count                    52285 # cumulative LSQ occupancy
LSQ_fcount                       0 # cumulative LSQ full count
lsq_occupancy               0.5403 # avg LSQ occupancy (insn's)
lsq_rate                    0.4818 # avg LSQ dispatch rate (insn/cycle)
lsq_latency                 1.1213 # avg LSQ occupant latency (cycle's)
lsq_full                    0.0000 # fraction of time (cycle's) LSQ was full
```

We observe that there is an improvement of 0.4778 – 0.4756 = 0.0022 IPC when we switch from compiler flag -O0 to optimization -O2.
Also, there is no further improvement (for the given code) when we then switch to -O3 from -O2.

Possible reasons for improvement:

- O2 flag requires lower number of instructions compared to O0. Hence, there is a some optimization here.
- Total number of branches, loads, stores executed are also less.
- Instruction rescheduling performed by -O2 also reduces the number of stalls.
- Also, it uses aggressive loop unrolling.
- Reduction of redundant load/stores by storing vales in registers instead of memory locations (whenever possible).